

# Aspirova

*Every opportunity. One place.*

A complete field guide to how this system was built, what broke, why it broke, and what each failure taught. Written to be read end to end - not skimmed.

|                                                 |                                               |                                            |
|-------------------------------------------------|-----------------------------------------------|--------------------------------------------|
| <div>26,883</div> <div>OPPORTUNITIES LIVE</div> | <div>4,769</div> <div>COMPANIES TRACKED</div> | <div>16</div> <div>SOURCES CRAWLING</div>  |
| <div>516</div> <div>COMMITTS IN 61 DAYS</div>   | <div>693</div> <div>BACKEND TESTS</div>       | <div>68,552</div> <div>LINES OF CODE</div> |

Solo-built by Sohan Mandal - 23 June to 25 August 2026  
Architecture by Claude Opus - implementation by Claude Sonnet and OpenAI Codex  
Status: live in production, pre-revenue, 10 registered users

## How to read this

This document has one job: to make you the person who understands Aspirova best. Not the person who remembers that it exists - the person who could rebuild it, defend every decision in it, and explain why each of its scars is there.

It is organised as a story, because that is how the system was actually built - one decision at a time, several of them wrong, each corrected by a measurement. The most valuable section is **Part IV, the war stories**. Anyone can read an architecture diagram. Almost nobody can tell you what it cost to arrive at one.

| Part                        | What it gives you                                                       |
|-----------------------------|-------------------------------------------------------------------------|
| <b>I - Origins</b>          | Where the idea came from, and the one insight the whole thing rests on. |
| <b>II - The Machine</b>     | What runs where, and how a request and a crawl actually flow.           |
| <b>III - By the Numbers</b> | The real production state, measured the day this was written.           |
| <b>IV - War Stories</b>     | Nine failures, with cause, fix, and the number that proved it.          |
| <b>V - The Laws</b>         | Twelve rules paid for in downtime and blown quotas.                     |
| <b>VI - Where It Stands</b> | Honest status, open risks, and what comes next.                         |

### HOW TO TEST YOURSELF

At the end of each part there is a **Check yourself** box. Answer out loud before reading on. If you cannot answer one, that part has not landed yet - go back. The answers are all in the preceding pages.

## PART I

# Origins

Every project has a founding insight - the one call that shapes everything after it. Aspirova has exactly one, and it was made on day one.

## What Aspirova is

Aspirova is an **AI-powered career-intelligence platform for students** that automatically discovers opportunities - internships, jobs, hackathons, fellowships - from public sources.

### THE DISTINCTION THAT DEFINES IT

Aspirova is **not a job board**. Companies never post here. Every listing is discovered by a crawler from a public source and linked back to its origin. A job board's supply problem is sales; Aspirova's is engineering. That single difference explains why this is a crawler company wearing a website.

## The founding insight

The obvious way to build this is to scrape the aggregators everyone already knows - Internshala, Unstop, Wellfound, LinkedIn. It is also the way that kills the company.

All of them forbid scraping in their terms and actively block bots. Building on them is legally fragile and technically a treadmill: every layout change is an outage. The durable alternative is to skip the middleman entirely.

### GO TO THE SOURCE, NOT THE AGGREGATOR

Companies publish their own openings through applicant tracking systems - **Greenhouse, Lever, Ashby, SmartRecruiters** - and those systems expose structured, semi-public JSON endpoints per company. That is where the majority of real, fresh, deduplicated data should come from, at near-zero cost and near-zero legal risk. Aggregators become a secondary, best-effort tier.

This decision, logged 23 June 2026, reshaped the entire crawler architecture. Today it is vindicated by the numbers: **21,174 of 26,883 listings** come from direct ATS endpoints - 79% of the corpus.

## The five things the canon rests on

Written before a line of product code, and all five still hold:

- **Go to the source.** ATS endpoints are the moat; aggregators are a bonus tier.

- **Crawl once, store centrally, serve many - and that must extend to AI.** The expensive trap is per-user AI. Each canonical opportunity is enriched exactly once, after deduplication, and every user reads the shared result for free.
- **Postgres is the search engine, the vector store and the queue.** Full-text, fuzzy matching, semantic search and job queueing in one managed database. No Elasticsearch, no Pinecone, no RabbitMQ - at this scale those are liabilities.
- **Resume Match is cosine similarity, not an LLM call.** Embeddings are computed once per opportunity; matching a resume is arithmetic, not inference.
- **Be honest about the money.** At Rs. 39 / Rs. 49 per month, break-even is around 1,000 users at a sober 3% conversion. This is currently a project, not a business - a correct build matters more than profit.

## The timeline

| When        | Milestone                                                                      |
|-------------|--------------------------------------------------------------------------------|
| 23 Jun 2026 | Canon written. ATS-first crawling adopted. Postgres-only data plane.           |
| 25 Jun 2026 | First commit. Pricing set at Rs. 39 / Rs. 49.                                  |
| 30 Jun 2026 | Phase 1 passes architect review. Deployed live - Render, Vercel, Supabase.     |
| 5 Jul 2026  | Phase 2 complete: rate limiting, caching, backups, email.                      |
| 6 Jul 2026  | Premium UI shipped. AI phase built. Codex joins as second implementer.         |
| 7 Jul 2026  | SEO content engine and referral loop land. 4th ATS adapter added.              |
| 8 Jul 2026  | Resume Match goes live. Razorpay wired but dormant behind a flag.              |
| 31 Jul 2026 | Programmes Registry architected - curated research fellowships.                |
| 3 Aug 2026  | First Vercel quota breach. 4h12m against a 4h CPU limit.                       |
| 9 Aug 2026  | Three Vercel meters exceeded simultaneously. The worst day on record.          |
| 14 Aug 2026 | Root cause found after eleven days: one line in lib/api.ts.                    |
| 16-18 Aug   | Auth defects, a meaningless filter, and silent crawler truncation - all fixed. |
| 24-25 Aug   | Browse-page caching. Every page cacheable. This document written.              |

### CHECK YOURSELF

1. Why is scraping Internshala a worse foundation than crawling Greenhouse?
2. What does 'crawl once, serve many' protect you from, in rupees?
3. Why is Resume Match arithmetic rather than an AI call?

## PART II

# The Machine

Six services, four providers, one database. Here is exactly what runs where, and what happens when someone loads a page.

## What runs where

| Layer              | Technology                       | Hosted on                      | Notes                                                    |
|--------------------|----------------------------------|--------------------------------|----------------------------------------------------------|
| <b>Frontend</b>    | Next.js 16.2.9                   | Vercel (Hobby, free)           | aspirova.vercel.app. Server-rendered for SEO.            |
| <b>Backend API</b> | FastAPI (Python 3.12)            | Render, Singapore (\$7/mo)     | aspirova-api.onrender.com. Closest region to the DB.     |
| <b>Database</b>    | Postgres 15 + pgvector           | Supabase, Mumbai (ap-south-1)  | Search, vectors and queue all in one box.                |
| <b>Crawlers</b>    | Python workers                   | GitHub Actions cron            | Runs 01:00 UTC daily. Free minutes - the repo is public. |
| <b>Cache</b>       | In-process L1 + Upstash Redis L2 | Upstash (free)                 | L1 is per-instance; L2 is shared across instances.       |
| <b>Email</b>       | Resend                           | Resend, Tokyo (ap-northeast-1) | Sending only. Inbound is handled by ImprovMX.            |
| <b>Backups</b>     | Nightly pg_dump                  | Cloudflare R2                  | Chosen for zero egress fees.                             |
| <b>Payments</b>    | Razorpay                         | Dormant                        | Fully wired in code, switched off behind a flag.         |

### WHY SINGAPORE AND MUMBAI, NOT ONE REGION

Render has no Mumbai region, so the API sits in Singapore roughly 120ms from the database in Mumbai. That hop was measured rather than assumed, and judged acceptable: the read path is cached, so most requests never touch the database at all. Consolidating regions remains a deferred option, not an urgent one.

## What happens when someone loads a page

Follow a single visit to the homepage, in order:

| # | Step              | What actually happens                                                                                                                             |
|---|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Browser -> Vercel | Vercel serves a <b>prerendered</b> HTML page from its edge cache. No server render, no function invocation, no database query.                    |
| 2 | Cache decision    | If the page is older than its 6-hour window, Vercel serves the stale copy immediately and regenerates in the background. The visitor never waits. |

| # | Step             | What actually happens                                                                                                             |
|---|------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 3 | Vercel -> Render | Only on regeneration. The Next.js server calls the FastAPI backend for the feed.                                                  |
| 4 | L1 cache         | The API checks an in-process cache first. A hit costs roughly zero and returns in single-digit milliseconds.                      |
| 5 | L2 cache         | On an L1 miss it tries Upstash Redis, shared across API instances - so a cold instance can still be warm.                         |
| 6 | Postgres         | Only on a full miss. The feed query is one round trip to Mumbai; total server execution measured at roughly 348ms.                |
| 7 | Response         | JSON returns to Next.js, which renders HTML and hands it back to Vercel's cache, where it serves everyone for the next six hours. |

#### THE POINT OF ALL THAT MACHINERY

Steps 3 through 7 run **about four times a day**, not once per visitor. That is the whole design: the expensive path is amortised across every person who visits in the window. Getting this wrong is what caused the biggest incident in the project's history - see Case 1.

## How a crawl works

The crawler is the actual product. Everything else is a window onto what it collects.

| # | Stage            | What happens                                                                                                                                                                                             |
|---|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Trigger          | GitHub Actions fires on cron at 01:00 UTC, or on manual dispatch.                                                                                                                                        |
| 2 | Source selection | Sources are ordered stalest-first, each with its own time budget and a reserved floor for the sources queued behind it.                                                                                  |
| 3 | Fetch            | Each adapter implements one interface - fetch, parse, health, coverage. 18 adapters exist: Greenhouse, Lever, Ashby, SmartRecruiters, Amazon, Unstop, Devpost, Devfolio, RemoteOK, HackerEarth and more. |
| 4 | Normalise        | Raw payloads become a common shape: title, company, location, country, deadline, category. This is where most historical bugs have lived.                                                                |
| 5 | Deduplicate      | The same role from two sources collapses into one canonical opportunity.                                                                                                                                 |
| 6 | Persist          | Bulk upserts into Postgres. Listings seen again get their last_seen_at stamped in a single set-based statement, not one query per row.                                                                   |
| 7 | Revalidate       | The crawler POSTs changed slugs and list paths to the frontend, which invalidates exactly those cached pages. Fresh content without waiting out the cache window.                                        |
| 8 | Report           | Every source reports coverage against what the source itself declares. A source that declares no total reports UNKNOWN - never 100%.                                                                     |

## The three mandated seams

Governance names three chokepoints that all related work must route through. They exist so a rule can be enforced in one place instead of audited everywhere:

- **ai\_client** - every AI call goes through it. One place to cap spend, swap models, or switch the whole system off.
- **can(user, feature)** - the single gate for paid features. One function decides what any user may do, so a pricing change touches one file.
- **SourceAdapter** - every crawler implements the same interface, so adding a source is writing one class, not editing the crawl loop.

#### CHECK YOURSELF

1. How many times a day does the database actually serve the homepage?
2. Why does the crawler run on GitHub Actions instead of the Render server?
3. What does the revalidation webhook buy that a shorter cache window would not?

## PART III

# By the Numbers

Everything below was measured against production on 25 August 2026 - queried, not remembered.

## The corpus

| Metric             | Value  | Reading                                                      |
|--------------------|--------|--------------------------------------------------------------|
| Opportunities live | 26,883 | All currently active.                                        |
| Companies tracked  | 4,769  | Mostly resolved dynamically from aggregator listings.        |
| Sources enabled    | 16     | Across ATS boards and aggregators.                           |
| Registered users   | 10     | Pre-launch. The product works; distribution has not started. |

## What is in the corpus

| Category     | Count  | Share |
|--------------|--------|-------|
| Jobs         | 23,968 | 89.2% |
| Internships  | 2,238  | 8.3%  |
| Hackathons   | 355    | 1.3%  |
| Competitions | 322    | 1.2%  |

## Where it comes from

| Source          | Listings | Type                                     |
|-----------------|----------|------------------------------------------|
| Greenhouse      | 13,674   | ATS - direct from company boards         |
| Ashby           | 5,303    | ATS - direct                             |
| Unstop          | 3,298    | Aggregator - competitions and hackathons |
| Lever           | 1,366    | ATS - direct                             |
| RemoteOK        | 953      | Aggregator - remote roles                |
| SmartRecruiters | 831      | ATS - direct                             |
| Amazon          | 493      | Proprietary board                        |
| Himalayas       | 393      | Aggregator - remote roles                |

**READ THIS TABLE AGAIN**

**21,174 of 26,883 listings - 79% - come from direct ATS endpoints.** That is the 23 June founding insight, proved in production two months later. The aggregators everyone else builds on contribute under a fifth of the corpus, and they are the tier that can be lost without killing the product.

## The codebase

| Metric              | Value                                                                |
|---------------------|----------------------------------------------------------------------|
| Commits             | 516, between 25 June and 25 August 2026                              |
| Backend             | 240 Python files, 47,098 lines                                       |
| Frontend            | 157 TypeScript files, 21,454 lines                                   |
| Backend tests       | 693                                                                  |
| Database migrations | 26                                                                   |
| Crawler adapters    | 21 modules covering 18 live sources                                  |
| Contributors        | 1 human, 3 AI agents (Opus architect, Sonnet and Codex implementers) |

**THE NUMBER WORTH SITTING WITH**

516 commits in 61 days is about 8.5 per day, every day, including the days spent hunting a single line of caching configuration. The project's velocity was never the constraint - **knowing whether a change actually worked** was. Almost every law in Part V is a variation on that theme.

## PART IV

# War Stories

Nine real failures. Each is laid out the same way: what it looked like, what was actually wrong, what fixed it, the number that proved it, and the rule it bought. This is the part worth re-reading.

## CASE 1

## The Vercel quota catastrophe

| Stage                  | What happened                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>         | On 3 August the free-tier CPU meter read <b>4h12m against a 4h monthly limit</b> . Projects auto-pause past the limit and upgrading was not an option. Six days later it was worse: <b>three meters breached at once</b> - ISR writes 434K/200K, CPU 6h15m/4h, origin transfer 9.82GB/10GB.                                                                                                        |
| <b>Root cause</b>      | It took <b>eleven days and seven failed fixes</b> to find. A page declared a 7-day cache window, but a single line in <code>lib/api.ts</code> set <code>revalidate: 60</code> on the fetch inside it. <b>A route's cache window is the minimum of its own setting and every fetch it makes</b> . The real window was 60 seconds across 20,427 pages - near-total regeneration on every crawl pass. |
| <b>The fix</b>         | Corrected the fetch value, widened windows, cut the sitemap from 20,793 URLs to 5,000, deleted per-item OG image routes, and added on-demand revalidation so pages refresh when content actually changes rather than on a timer.                                                                                                                                                                   |
| <b>Proof it worked</b> | Cache headers went from STALE within a minute to HIT still climbing at Age: 411. False revalidations fell from <b>31% to 12%</b> of listings per crawl.                                                                                                                                                                                                                                            |

### THE LAW THIS BOUGHT

**A guardrail written from a guess is indistinguishable from a bug.** Every work order during this incident said 'do not change revalidate in `lib/api.ts`' - fencing off, on an unverified assumption, the exact line defeating the fix. Never protect a value you have not proven inert.

## CASE 2

## The crawler that lied about succeeding

| Stage             | What happened                                                                                                                                                                                                                                                                                                          |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>    | Every crawl reported SUCCESS. Meanwhile Unstop was saving <b>55 of roughly 1,100</b> available opportunities, Devpost and RemoteOK had not run for <b>11 days</b> , Amazon for 7, and Unstop had not completed a single clean run in <b>37 days</b> .                                                                  |
| <b>Root cause</b> | A <b>single shared 600-second budget</b> across all aggregators. Unstop alone exceeded it, and the loop's break statement skipped every source queued behind it. The stalest-first ordering that existed to prevent exactly this could not help, because one source consumed the entire pool before the sort mattered. |

| Stage                  | What happened                                                                                                                                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The fix</b>         | Budgets became per-source: the running source takes everything remaining except a floor reserved for each source behind it, with that floor itself capped at an even split. Coverage is now measured against what each source declares. |
| <b>Proof it worked</b> | An even split was measured and <b>rejected</b> - it gave all three aggregators 2,900s, truncating Unstop at 95.4% while Devpost finished in 130s and RemoteOK in 240s.                                                                  |

#### THE LAW THIS BOUGHT

**A truncated crawl looked identical to a complete one.** Success is not a status code, it is a coverage number - and a source that declares no total must report UNKNOWN, never 100%. An unverifiable completeness claim is what let this hide for weeks.

### CASE 3

## The auth that asserted what it never checked

| Stage                  | What happened                                                                                                                                                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>         | Signup told users 'We sent a confirmation link' when no email had been sent. Separately, <b>no password reset existed anywhere</b> - any user who forgot their password was locked out permanently. Two real users had signed up the day this was found.         |
| <b>Root cause</b>      | Supabase answers signup on an already-registered address with <b>success and a decoy user</b> , deliberately, so signup cannot be used to enumerate accounts. The UI read that as a real signup. The decoy is distinguishable only by an empty identities array. |
| <b>The fix</b>         | Inspect the returned payload rather than the absence of an error. Shipped password reset, and added a 'resolving' state so a working reset link is never reported as expired mid-exchange.                                                                       |
| <b>Proof it worked</b> | Verified against production Supabase, not against the documentation.                                                                                                                                                                                             |

#### THE LAW THIS BOUGHT

**An auth surface may not assert an outcome it did not inspect.** Anti-enumeration APIs deliberately return success for failure. Also: 'resolving' is a third state, distinct from present and absent, on every auth-gated surface.

### CASE 4

## The filter that returned plausible nonsense

| Stage             | What happened                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>    | A request for India/Abroad filters on competitions. Built literally, India returned 110 results and Abroad returned 110 - <b>sharing 106 rows</b> . Both numbers looked entirely reasonable on screen.                                                                                                                                                                                                          |
| <b>Root cause</b> | The bug was upstream in ingestion, not the UI. <code>crawlers/unstop.py</code> assigned <code>location = region</code> , and <code>region</code> only ever holds 'online' or 'offline' - <b>a modality stored where a place belongs</b> . So country detection could never resolve anything: <b>0 of 71</b> Unstop competitions had a country, while the payload had carried city, state and country all along. |
| <b>The fix</b>    | Fixed the adapter and backfilled. Competitions with a country went 8 -> 43, India-only 4 -> 38. Online became its own filter option rather than being hidden inside a geographic choice.                                                                                                                                                                                                                        |

| Stage                  | What happened                                                                         |
|------------------------|---------------------------------------------------------------------------------------|
| <b>Proof it worked</b> | <b>0 rows lost a country or a real location</b> - checked before applying, not after. |

**THE LAW THIS BOUGHT**

**Verify a filter against the actual data distribution before building it.** A control that returns plausible numbers while being meaningless is worse than no control at all. And 'no country' is sometimes correct data, not missing data.

**CASE 5****87GB of bandwidth nobody was using**

| Stage                  | What happened                                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>         | Render reported 87GB of 'service-initiated' bandwidth - far beyond anything the traffic could explain.                                  |
| <b>Root cause</b>      | A 45-second cache TTL applied to very large payloads. The sitemap alone was 1.69MB and was being regenerated and re-shipped constantly. |
| <b>The fix</b>         | Tiered cache TTLs matched to payload size, plus explicit Cache-Control headers.                                                         |
| <b>Proof it worked</b> | Bandwidth fell to 3-7GB/month, verified over a full day of traffic.                                                                     |

**THE LAW THIS BOUGHT**

**Cache policy must be proportional to payload size.** A short TTL that is harmless on a 2KB response is ruinous on a 1.7MB one.

**CASE 6****The cache that failed on every single request**

| Stage                  | What happened                                                                                                                                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>         | The Upstash L2 cache had been failing on every call since 18 August, printing a full traceback each time. A correctness fix had stopped it breaking responses - but not stopped it being <b>called</b> .                            |
| <b>Root cause</b>      | No circuit breaker. Every uncached request made two doomed network calls to a quota-exhausted Redis, each with a 1.5-second timeout. A genuinely hanging Upstash would have added <b>3 seconds to every request</b> , indefinitely. |
| <b>The fix</b>         | A circuit breaker: after 3 consecutive failures, skip L2 entirely for 300 seconds, then send one probe. Log once per cooldown so a long outage stays visible.                                                                       |
| <b>Proof it worked</b> | In production, 19 seconds after deploy: breaker opened, then logged again at exactly 300 seconds. Traceback spam went to zero.                                                                                                      |

**THE LAW THIS BOUGHT**

**Fixing the symptom is not fixing the cost.** The earlier fix made the failure harmless but left it happening thousands of times a day. Ask not only 'is it broken' but 'what is it still costing'.

**CASE 7**

## The adapter that never ran

| Stage                  | What happened                                                                                                                                                                                                                                                                      |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>         | A Devfolio crawler was written, reviewed, merged and deployed. Weeks later the database contained <b>zero</b> Devfolio opportunities.                                                                                                                                              |
| <b>Root cause</b>      | The adapter code shipped, but no row was ever created in the sources table. The crawler iterates over enabled sources in the database - with no row, the adapter was <b>unreachable code in production</b> . Nothing in CI could catch it, because the code was perfectly correct. |
| <b>The fix</b>         | Registered the source with the settings mirrored exactly from a working aggregator, then ran the adapter directly against the live API before trusting a crawl.                                                                                                                    |
| <b>Proof it worked</b> | 32 hackathons ingested on the next crawl - <b>25 of them India-based</b> .                                                                                                                                                                                                         |

### THE LAW THIS BOUGHT

**Shipped is not the same as running.** A feature that depends on configuration is not delivered until that configuration exists in production. Green tests prove code compiles; only data proves it runs.

### CASE 8

## The pages that could not be cached

| Stage                  | What happened                                                                                                                                                                                                                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b>         | The Vercel CPU budget was exhausted again - <b>aspirova alone was 99.2%</b> of a team-wide limit. Five browse pages were rendering on <b>every single visit</b> .                                                                                                                                        |
| <b>Root cause</b>      | A Next.js page that reads searchParams can never be statically rendered. Pagination lived in <code>?page=N</code> , which forced per-request rendering. Removing <code>force-dynamic</code> - the obvious fix - had already been tested and <b>disproved</b> : it is a no-op while searchParams is read. |
| <b>The fix</b>         | Moved pagination into the URL path ( <code>/jobs/page/2</code> ). For the homepage and competitions, whose filters are intrinsic, the page became static with a client-side takeover for filtered views.                                                                                                 |
| <b>Proof it worked</b> | All five pages went from per-visit rendering to a 6-hour cache. The homepage still ships <b>273KB of real HTML with 38 live listings</b> - SEO fully intact.                                                                                                                                             |

### THE LAW THIS BOUGHT

**The obvious fix had already been tried and disproved - and was still in reach.** Write down what you have ruled out, or you will pay for the same experiment twice.

### CASE 9

## The redirect that was never doing the work

| Stage          | What happened                                                                                                                                                             |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Symptom</b> | After shipping the caching fix, <code>/jobs/page/1</code> returned a 404 page with a 200 status. The identical pattern on an older route returned a correct 308 redirect. |

| Stage           | What happened                                                                                                                                                                                                                                                                                                            |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Root cause      | Both routes called <code>redirect()</code> in the component - so the component was never the difference. The older route worked only because <code>next.config.ts</code> carried a redirect rule for it. <b>The in-component redirect does not fire once a route is ISR-prerendered.</b> Half a pattern had been copied. |
| The fix         | Added the missing config-level rules for the new routes.                                                                                                                                                                                                                                                                 |
| Proof it worked | All three returned 308 in production, confirmed by request rather than by build output.                                                                                                                                                                                                                                  |

#### THE LAW THIS BOUGHT

**When you copy a working pattern, verify you copied the part that does the work.** This was found only by curling production - the build was green and would never have revealed it.

#### THE THREAD RUNNING THROUGH ALL NINE

Not one of these was caught by tests. Every one shipped with lint, types, tests and build fully green. Six of the nine were found only by **measuring production** - a cache header, a row count, an HTTP status. That is why strict verification became a merge gate on 17 August, and it is the single most important operational lesson in this document.

## PART V

# The Laws

---

Twelve rules, each paid for with real downtime, a blown quota, or a user-visible defect. They are listed in the order they were learned.

**01 Green gates prove a change compiles, never that it works.**

This project has shipped, with every gate green, a pipeline producing 5% lists, a message rendering 'user@gmail.com is already registered', and a filter returning 110-vs-110 overlapping rows. Exercise against real production data before merge.

**02 Measure the mechanism, not the symptom.**

During the Vercel incident the diagnosis was wrong twice from reasoning over aggregate dashboards. The answer sat in the ISR writes panel the whole time.

**03 A fetch's cache setting silently caps its whole route's cache window.**

The route window is the minimum of its own value and every fetch inside it. This single line cost eleven days.

**04 Never protect a value you have not proven inert.**

Work orders fenced off the exact line causing the incident, based on an assumption.

**05 An interface may not assert an outcome it did not inspect.**

Anti-enumeration APIs deliberately return success for failure. The payload is the evidence; the absence of an error is not.

**06 'Resolving' is a third state, distinct from present and absent.**

Treating 'not yet known' as 'known to be absent' told users their working links had expired.

**07 Verify a filter against the real data distribution before building it.**

A control returning plausible but meaningless numbers is worse than no control.

**08 Coverage is measured against what the source declares - no total means UNKNOWN.**

A truncated crawl is indistinguishable from a complete one without this.

**09 A summed per-query total is not a valid denominator when queries overlap.**

Amazon reported 89.7% by summing 485 across overlapping searches against 435 deduped.

**10 Deploy frequency is itself a cost driver.**

Every production deploy invalidates the entire cache. Three deploys in 56 minutes produced the worst day on record - while shipping cost fixes. A day containing a deploy cannot measure a caching change.

**11 Shipped is not running.**

A feature depending on configuration is not delivered until that configuration exists in production.

**12 When a measurement disagrees with a prediction, suspect the measurement first.**

Two architect queries were themselves wrong in a single session - a case-insensitive check that flagged its own target, and a one-to-many join inflating 1,216 rows to 1,329.

## PART VI

# Where It Stands

An honest status report: what is done, what is fragile, and what is genuinely unknown.

## Working and verified

- **Discovery.** 16 sources, 26,883 live opportunities, per-source coverage measured and reported rather than assumed.
- **Read path.** Every browse page cached at a 6-hour window with server-rendered HTML intact for search engines.
- **Freshness.** The crawler pushes invalidations for exactly the pages that changed.
- **Email.** Daily digests and weekly reports delivering to real users; domain fully verified for sending.
- **Resilience.** Rate limiting, two-tier caching with a circuit breaker, nightly off-site backups with restore actually tested.

## Known and open

| Item                            | Status                    | Note                                                                                                                                            |
|---------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vercel CPU budget</b>        | Unproven                  | Every browse page is now cached, but the meter needs a full day without a deploy before the fix can be called proven.                           |
| <b>Detail-page regeneration</b> | Unquantified              | Opportunity pages are roughly 78% of function invocations. Whether that is deploy churn or a scraper walking the corpus has not been separated. |
| <b>A bot walking the corpus</b> | Observed, unaddressed     | Something requests deep pages sequentially, about one page per hour. It is the only consumer of deep pagination.                                |
| <b>Soft 404s</b>                | Mitigated, not eliminated | Out-of-range pages return 200 while rendering a not-found page. Marked noindex so they are not indexed; a true 404 needs a framework change.    |
| <b>Distribution</b>             | Not started               | 10 registered users. The product works; nobody has been told about it.                                                                          |
| <b>Revenue</b>                  | Dormant by choice         | Razorpay is fully wired behind a flag. Break-even needs roughly 1,000 users.                                                                    |

## If you remember only one page

Make it this one.

**THE SHAPE OF ASPIROVA IN SIX SENTENCES**

It is a crawler wearing a website. Its moat is that it goes to company ATS endpoints directly instead of scraping aggregators, which is why 79% of the corpus is legally clean and structurally stable. Everything expensive is done once and shared - crawling, enrichment, embeddings, caching - because per-user work is what makes this class of product unaffordable. One Postgres database is the search engine, the vector store and the queue, and that is a deliberate refusal to add infrastructure. Every serious failure in its history came from believing something without measuring it. And the fix, every single time, was a number pulled from production.

Aspirova Docs - compiled 25 August 2026 from the docs/ canon, the git history, and live production measurements.